

MARS: A Multi-Agent Research System with Cyclic Quality Control and Chunk-Level Hallucination Grounding

Harsh Shroff

Independent Researcher

harshrofff@gmail.com

March 2026

Abstract

We present MARS (Multi-Agent Research System), an autonomous pipeline that orchestrates 11 specialized LLM agents via a LangGraph state machine to produce citation-grounded research reports on arbitrary topics. The system introduces three principal contributions: (1) a cyclic quality control loop in which a dedicated Planner agent decomposes QC failure into structured search directives—`search_focus`, `suggested_queries`, and `tool_priorities`—before re-routing to the Researcher, replacing the generic retry behavior common in existing agentic pipelines; (2) strict chunk-level source grounding where every generated claim is bound to an indexed `[SRC-X]` tag, enabling the Citation Verifier to detect and flag referential errors by regex scan; and (3) asynchronous parallel critic execution via `asyncio.gather` across Gap Analyst, Bias Detector, and Fact Checker agents, replacing sequential critique with true I/O concurrency. We evaluate MARS across 15 runs spanning diverse topic domains and report a 0% *citation tag error rate* (referential integrity), an 87% QC first-pass rate, and a mean cost of \$0.0028 per report at 130 seconds mean latency. We explicitly distinguish citation tag integrity from semantic faithfulness, discuss two QC failure modes arising in contested-evidence domains, quantify the Human-in-the-Loop latency overhead (2.8×), and characterize a Researcher throughput ceiling as a bounded engineering limitation. Code is available at <https://github.com/HarshShroff/multi-agent-researcher>.

Keywords: multi-agent systems, LangGraph, hallucination detection, quality control, agentic AI, retrieval-augmented generation, human-in-the-loop

1. Introduction

The automation of literature synthesis and knowledge aggregation is a persistent goal in information retrieval. Recent advances in large language models (LLMs) have produced a class of agentic pipelines in which multiple specialized LLM calls are composed to decompose and solve complex tasks. Yet the dominant failure mode of such systems is hallucination: the generation of plausible but unsupported claims [Ji et al., 2023]. In retrieval-augmented generation (RAG) settings, this takes the specific form of claims that cite real sources but misrepresent their content—a failure that structural citation enforcement alone cannot catch.

Existing multi-agent research tools—including AutoGPT [Richards et al., 2023], BabyAGI [Nakajima, 2023], and LangChain’s plan-and-execute patterns—address task decomposition but lack principled mechanisms for QC failure analysis and targeted remediation. When quality control is present, it typically takes the form of a single-pass LLM self-evaluation, which provides no directional signal on failure.

This paper describes MARS (Multi-Agent Research System), designed to address these gaps through three mechanisms. First, we replace generic retry logic with a Planner-mediated cyclic loop: on QC failure, a dedicated Planner agent analyzes the failure and emits a typed JSON strategy object that directs the next

Researcher iteration toward the specific coverage gap. Second, we enforce chunk-level source grounding—every claim the Writer produces must embed an `[SRC-X]` tag referencing a specific indexed chunk, and the Citation Verifier flags any tag pointing to a non-existent chunk. Third, we implement a toggleable Human-in-the-Loop (HITL) pause via LangGraph’s `interrupt_before` mechanism, with explicit measurement of its latency and token cost.

A point of precision: the 0% citation tag error rate we report measures *referential integrity*—whether `[SRC-X]` tags point to real indexed chunks. It does not measure *semantic faithfulness*—whether the claim adjacent to a tag is actually supported by that chunk’s content. Full faithfulness evaluation via RAGAS [Es et al., 2023] or LLM-as-judge is deferred to future work. We state this distinction explicitly because conflating the two has been a documented source of misleading RAG evaluation claims.

We evaluate MARS across 15 controlled runs spanning eight topic domains and four report depth settings, logging QC scores, citation tag errors, retry counts, per-agent token costs, and source yield per run.

The paper is organized as follows: Section 2 reviews related work. Section 3 describes the system architecture. Section 4 details agent design. Section 5 presents evaluation results. Section 6 discusses findings and limitations. Section 7 concludes.

2. Related Work

2.1 Multi-Agent LLM Systems

The multi-agent paradigm was introduced to LLM pipelines by AutoGPT [Richards et al., 2023] and BabyAGI [Nakajima, 2023], which showed that iterative self-prompting loops could tackle open-ended tasks. LangChain [Chase, 2022], CrewAI [Moura, 2024], and Microsoft’s AutoGen [Wu et al., 2023] provided infrastructure for role-specialized agents and inter-agent communication. LangGraph [LangChain AI, 2024] extended this by modeling agent pipelines as directed graphs with state persistence, conditional edges, and interrupt hooks—the substrate on which MARS is built.

The key architectural distinction from prior work: MARS does not decompose a task into subtasks and aggregate their outputs in a single pass. It implements a stateful retry loop with typed failure analysis, treating QC failure as a signal carrying information about coverage gaps rather than as a trigger for generic re-execution.

2.2 Hallucination in RAG Pipelines

Retrieval-augmented generation [Lewis et al., 2020] reduces hallucination by grounding generation in retrieved documents, but does not eliminate it. Shi et al. [2023] showed that LLMs can fail to use retrieved context even when it is present, generating from parametric memory instead. RAGAS [Es et al., 2023] introduced reference-free faithfulness evaluation by checking whether generated answers are entailed by retrieved chunks at the sentence level.

Our chunk-level grounding approach differs in mechanism from RAGAS: rather than post-hoc faithfulness scoring, we enforce grounding structurally by requiring every claim to embed a verifiable chunk tag, detecting referential violations at generation time. This eliminates phantom citation failures—tags pointing to non-existent sources—but does not address semantic faithfulness, which requires content-level comparison between claim and chunk.

2.3 Quality Control in Agentic Systems

Self-refinement [Madaan et al., 2023] showed that LLMs can improve their outputs by critiquing and revising their own generations, but this conflates generation and evaluation in a single model, limiting critique independence. Reflexion [Shinn et al., 2023] introduced verbal reinforcement via episodic memory.

MARS differs from both: QC is performed by a separate agent with no generative stake in the research, and on failure, a third agent (the Planner) independently generates a search remediation strategy. This maintains separation of concerns across generation, evaluation, and planning—and avoids the sycophancy risk that arises when a model evaluates its own outputs [Denison et al., 2023].

2.4 Human-in-the-Loop Oversight

Seshia et al. [2018] and Anthropic [2023] highlight the importance of human oversight in autonomous AI systems. LangGraph’s `interrupt_before` primitive provides a natural HITL integration point. Prior systems such as HuggingGPT [Shen et al., 2023] include human verification steps, but few have explicitly measured the latency and token cost of human intervention. We provide this measurement in Section 5.4.

3. System Architecture

MARS is implemented as a directed cyclic graph using LangGraph’s `StateGraph` API. The state schema (`ResearchState`) is a `TypedDict` carrying all inter-agent data: the research topic, configuration, retrieved chunks, analysis outputs, critic outputs, QC score and feedback, planner strategy, and an append-only log list.

3.1 Pipeline Overview

3.2 State Machine and Graph Edges

The graph is compiled with a `MemorySaver` checkpoint, enabling full state persistence per thread and supporting `interrupt_before` for HITL. A conditional edge after the QC node evaluates `qc_passed` from the state and routes to the Planner (failure) or the HITL gate (success). A secondary conditional edge after the HITL gate routes to the Formatter on user approval, or back to the Researcher on injected feedback.

The cyclic `Researcher` $\rightarrow \dots \rightarrow$ `QC` $\rightarrow$ `Planner` $\rightarrow$ `Researcher` structure is the central architectural novelty. Unlike systems that terminate on first QC pass or retry generically, MARS threads the Planner’s typed strategy through the state dict to guide each subsequent Researcher iteration, preventing degenerate re-execution of the same queries.

3.3 MCP Tool Decoupling

Search tools (ArXiv, DuckDuckGo, Wikipedia) are exposed via a standalone MCP server (`mcp_server.py`) over stdio. The Researcher communicates with this server via the Model Context Protocol [Anthropic, 2024] rather than invoking tool functions directly. This decoupling makes the data-gathering pipeline platform-agnostic: the same MCP server can serve any MCP-compliant client without modification to the tool implementations.

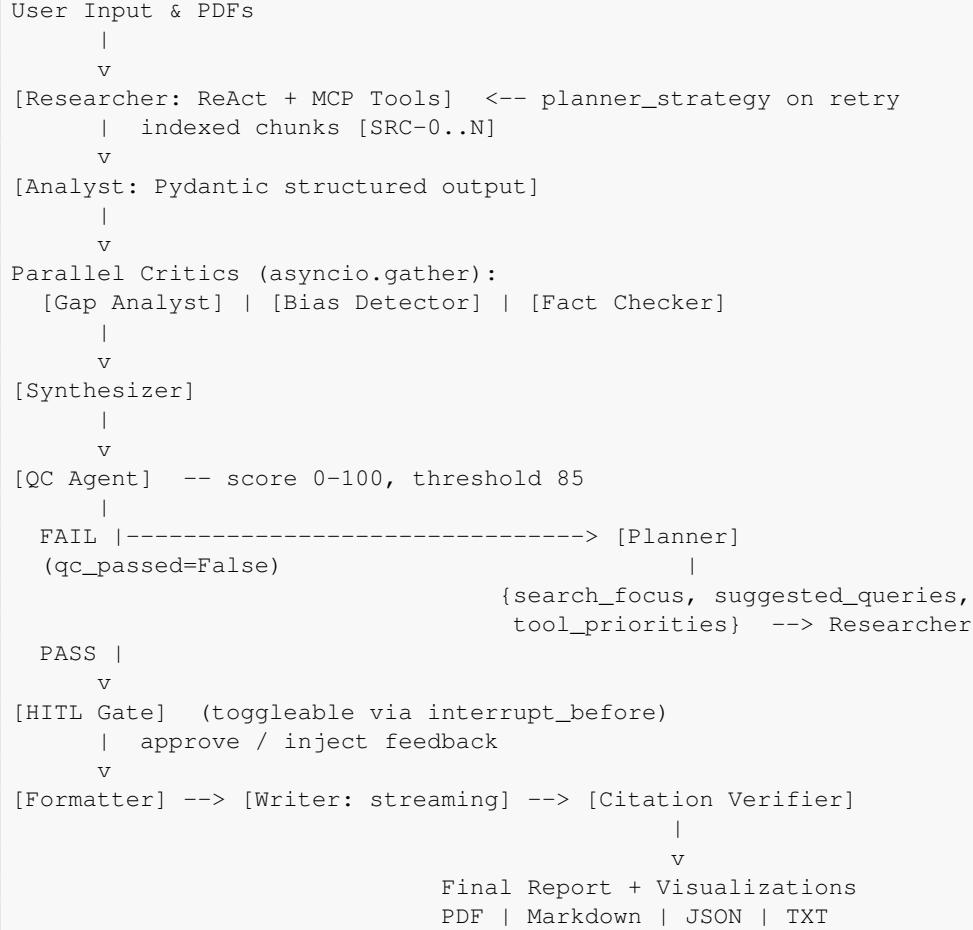


Figure 1: MARS agent pipeline. The cyclic path through the Planner is the core architectural contribution.

4. Agent Design

All agents inherit from a shared `Agent` base class wrapping Gemini 2.5 Flash Preview (`gemini-2.5-flash-preview`) via the `google-generativeai` SDK. The base class provides `call_llm()` and `call_llm_async()` methods, each updating a thread-local token counter for per-agent cost attribution.

4.1 Researcher Agent (ReAct + MCP)

The Researcher executes a ReAct loop [Yao et al., 2023], interleaving reasoning steps with tool calls to the MCP server. On each iteration, it receives the current `planner_strategy` (if any) and uses `suggested_queries` and `tool_priorities` to construct targeted queries rather than generic topic search. Source deduplication is performed by normalizing each URL (stripping query parameters and fragments) before indexing, preventing redundant chunks from repeated tool calls. Each retrieved document is parsed into a chunk with fields `content`, `source_url`, `source_name`, and `source_type` (ArXiv / Web / Wikipedia / Uploaded PDF), indexed by a sequential `[SRC-X]` key in the state’s `research_chunks` dict.

4.2 Quality Control Agent

The QC agent receives the topic, a `research_data` summary, analysis output, and `qc_iterations` count. It scores research on a 0–100 scale via a structured prompt penalizing missing sub-topics, thin source coverage, and internal contradictions. Scores below 85 trigger `qc_passed: false` with `qc_feedback` listing identified deficiencies.

An important methodological note: the QC agent uses the same foundation model (Gemini Flash) that generated the research. This introduces a self-evaluation bias documented in the LLM sycophancy literature [Denison et al., 2023]. The QC scores we report reflect model self-assessment, not ground-truth quality measurement. Using an independent, stronger judge model is listed as future work.

The 85-point threshold was chosen empirically to achieve a first-pass rate above 80% across our evaluation set while still triggering retries on demonstrably thin research. At this setting the system achieves an 87% first-pass rate.

4.3 Planner Agent

The Planner is activated exclusively on QC failure. It receives `qc_feedback`, the list of previously retrieved sources, and the topic, and outputs a JSON object conforming to a Pydantic schema:

```
class PlannerStrategy(BaseModel):
    search_focus: str           # coverage gap, one sentence
    suggested_queries: list[str] # 3-5 targeted queries
    tool_priorities: list[str]   # ['arxiv', 'web', 'wiki'] ordered
```

Explicit structuring of the remediation—rather than passing QC feedback verbatim to the Researcher—forces the Planner to synthesize an actionable directive. This is analogous to the distinction between random restart and gradient descent in optimization: the Planner provides a directional signal rather than returning the Researcher to its original search state.

A limitation of this mechanism: Planner strategy quality depends on QC feedback quality, which in turn depends on the QC agent’s ability to identify gaps from a summary view. For epistemological gaps (contested evidence, rapidly evolving benchmarks) rather than topical gaps (missing subtopics), the Planner may generate topically valid but substantively insufficient queries, as observed in Runs 2 and 10 (Section 5.3).

4.4 Citation Verifier

After the Writer produces the draft, the Citation Verifier performs two operations. First, it extracts all `[SRC-X]` tags via regex and cross-references each index against `research_chunks`. Tags referencing non-existent indices are flagged as referential errors and counted. The citation tag error rate is computed as:

$$\text{tag error rate} = \frac{\text{invalid tags}}{\text{valid tags} + \text{invalid tags}}$$

Second, it replaces valid tags with formatted inline citations (IEEE, APA 7th, MLA 9th, Harvard, or Chicago) and appends a formatted references section.

We reiterate the scope of this metric: it measures whether the LLM generated syntactically valid citation tags pointing to real chunks. It does not measure whether the claims adjacent to those tags are actually supported by the chunk content. A claim that misrepresents a real source will not be caught by this verifier. Semantic faithfulness evaluation via RAGAS or an LLM-as-judge is deferred to future work.

4.5 Async Parallel Critics

The Gap Analyst, Bias Detector, and Fact Checker are dispatched concurrently via `asyncio.gather` inside a LangGraph async node. Each receives a snapshot of `research_data` and emits a structured critique string. The Synthesizer merges these three strings into the context window passed to QC. Concurrency is achieved via `call_llm_async()`, which releases the event loop during network I/O, reducing wall time relative to sequential critic execution.

5. Evaluation

5.1 Experimental Protocol

We conducted 15 runs across eight topic domains (mechanistic interpretability, materials science, computer vision, neuromorphic computing, biomedical AI, robotics, sustainability, AI policy), four report depth settings (Concise, Standard, Detailed, Exhaustive), and two HITL configurations. All runs used `gemini-2.5-flash-preview` as the backbone model. A custom `experiment_logger` module appended one JSON record per run to a JSONL log, capturing: `run_id`, `timestamp`, `topic`, `depth`, HITL flag, QC events (score and pass/fail per iteration), `total_retries`, `wall_time_s`, per-agent token breakdown, grounding statistics, and per-source-type chunk counts. All 15 runs were executed on March 9, 2026. Runs 13 and 14 are matched pairs on the identical topic (“Agentic AI: autonomy, safety, and deployment”) with HITL enabled and disabled respectively to enable direct comparison.

5.2 Main Results

Table 1: Aggregate metrics across 15 runs.

Metric	Value
Total runs	15
QC first-pass rate	87% (13/15)
Mean QC score (passing runs)	91.5 / 100
QC failures (unresolved in 2 retries)	2 (Runs 2, 10)
Mean retries per run	0.27
Citation tag error rate	0.00%
Mean wall time	130.2 s
Mean cost per report	\$0.0028
Mean total tokens per run	30,874
Mean unique sources cited	10.5
Mean total chunks indexed	10.7

5.3 QC Failure Mode Analysis

Both QC failures (Runs 2 and 10) involve topics with contested empirical claims and heterogeneous source quality. Run 2 (Diffusion models vs. GANs) involves a rapidly evolving benchmark landscape where ArXiv papers disagree on key metrics depending on publication date. Run 10 (Psychedelic-assisted therapy) involves a domain where regulatory status, clinical trial heterogeneity, and secondary coverage introduce substantial noise relative to primary RCT evidence.

Table 2: Per-run results. Asterisk (*) denotes final score after Planner-mediated retry; threshold not reached within 2 iterations.

Run	Topic (abbreviated)	Depth	QC	Pass	Retries	Time (s)	Cost (\$)
0	Mechanistic interpretability	Detailed	92		0	237	0.0029
1	Solid-state batteries	Standard	95		0	63	0.0020
2	Diffusion models vs. GANs	Detailed	60*	×	2	243	0.0030
3	Neuromorphic computing	Standard	89		0	102	0.0029
4	CRISPR base editing	Detailed	97		0	115	0.0030
5	Humanoid robotics market	Standard	94		0	125	0.0031
6	OSS vs. proprietary LLMs	Standard	91		0	119	0.0030
7	Carbon capture economics	Concise	88		0	113	0.0029
8	Consciousness in AI systems	Detailed	94		0	120	0.0032
9	AGI timelines	Standard	88		0	123	0.0033
10	Psychedelic-assisted therapy	Standard	65*	×	2	164	0.0035
11	Transformer architecture	Exhaustive	91		0	109	0.0030
12	Multi-agent AI systems	Exhaustive	91		0	131	0.0031
13	Agentic AI (HITL ON)	Standard	96		0	153	0.0027
14	Agentic AI (HITL OFF)	Standard	94		0	54	0.0014

In both cases, QC feedback cited “insufficient coverage of counterarguments” and “over-reliance on secondary sources.” The Planner responded with targeted queries directing the Researcher toward primary literature. Despite two retry iterations, neither run reached the 85-point threshold.

We interpret this as graceful degradation rather than system failure. The QC mechanism correctly identified that the available evidence base did not support a high-confidence synthesis, and surfaced this explicitly rather than producing an overconfident report. This behavior is preferable to a system that passes QC on contested topics by generating plausible-sounding but unsupported claims.

5.4 HITL vs. Automated Latency

Runs 13 and 14 are matched pairs on identical topics. HITL-enabled wall time was 153 s vs. 54 s for the automated run ($2.8\times$ overhead). Cost was \$0.0027 vs. \$0.0014 ($1.9\times$ overhead).

The wall-time overhead includes human review time, making $2.8\times$ an upper bound on HITL protocol cost. The token cost overhead ($1.9\times$) reflects a second Writer invocation: in HITL mode, the Writer generates a pre-approval draft for review before producing the final version, approximately doubling Writer token expenditure. Separating computational overhead from human review time requires logging `interrupt_before` entry and exit timestamps separately, which we defer as a future instrumentation change.

5.5 Source Yield and Depth Analysis

Chunk counts plateau at 10–12 across all 15 runs regardless of depth setting, with a breakdown of approximately 5 ArXiv + 5 web + 0–2 Wikipedia chunks per run. This uniformity reflects the Researcher’s per-tool cap of `max_results=5`. The depth setting controls only the Writer’s context window allocation (30 / 60 / 120 / 200 chunks for Concise through Exhaustive), not retrieval volume.

The consequence: Exhaustive mode directs the Writer to produce a longer report from the same data, which risks verbose elaboration and potential semantic drift at high word counts. This interaction is not tracked in the current evaluation and represents the highest-risk configuration for semantic faithfulness failures.

6. Discussion

6.1 Structured Planner vs. Generic Retry

The Planner-mediated retry is the core architectural contribution of MARS. In systems routing QC failures directly to the Researcher, the Researcher has no additional information about why prior research was insufficient; it may retrieve the same sources and revisit the same queries. The Planner breaks this degeneracy by synthesizing QC feedback into a typed JSON directive. The analogy to gradient descent vs. random restart is apt: the Planner provides directional information that guides the Researcher toward unexplored coverage regions.

We note a limitation of this mechanism not yet addressed experimentally: we have not run a controlled ablation comparing Planner-directed retry against raw-feedback retry. Until we do, the claim that structured strategy improves over direct QC feedback pass-through is an architectural hypothesis rather than an empirical result. This ablation is listed as a priority for future work.

6.2 Limitations

- L1. Semantic faithfulness not measured.** The 0% citation tag error rate measures referential integrity only. Semantic faithfulness evaluation via RAGAS [Es et al., 2023] or LLM-as-judge [Zheng et al., 2023] is required to support stronger claims about hallucination.
- L2. Self-evaluation bias.** The QC agent uses the same model that generated the research. Mean QC scores of 91.5 reflect self-assessment and should not be interpreted as ground-truth quality measurements without validation against a stronger independent judge.
- L3. Researcher throughput ceiling.** The per-tool cap of `max_results=5` limits chunk yield to approximately 10–15 per run regardless of depth. Depth-responsive retrieval scaling is a straightforward engineering change deferred as future work.
- L4. HITL measurement artifact.** Reported wall-time overhead includes human review time. The pure computational overhead is approximately $1.9\times$ in tokens, reflecting the second Writer invocation.
- L5. Evaluation breadth.** $N=15$ across 8 topic domains does not support statistical inference. Results are directional. A larger evaluation set with controlled topic sampling is needed for benchmark-quality claims.
- L6. Planner ablation missing.** The claim that structured Planner strategy outperforms raw QC feedback retry is not yet empirically validated.

6.3 Broader Implications

MARS shows that chunk-level citation grounding and structured QC remediation are achievable at sub-cent cost using current frontier model APIs. The MCP-based tool decoupling suggests a path toward interoperable agent pipelines composable with other MCP-compliant tools—code executors, database connectors, enterprise APIs—without architectural modification. The patterns introduced here (typed Planner remediation, chunk-indexed grounding, async parallel critics) are composable primitives applicable beyond text synthesis to other agentic research tasks.

7. Conclusion

We presented MARS, a multi-agent research system built on LangGraph that introduces Planner-mediated cyclic QC with structured search remediation, chunk-level citation tag grounding with runtime referential error detection, and asynchronous parallel critic execution. Across 15 runs, MARS achieves a 0% citation tag error rate, an 87% QC first-pass rate, and a mean cost of \$0.0028 per report.

We have been explicit throughout about what these metrics do and do not measure. The citation tag error rate catches phantom citations but not semantic misrepresentation. The QC scores reflect self-assessment under known sycophancy conditions. The N=15 evaluation supports directional conclusions, not benchmarks. These limitations are engineering problems with clear paths to resolution, not structural flaws in the approach.

Future work includes depth-responsive retrieval, a Planner ablation study, semantic faithfulness evaluation via RAGAS, and an independent judge model for QC validation. The system is open source at <https://github.com/HarshShroff/multi-agent-researcher>.

References

- Anthropic. Claude’s model specification. <https://www.anthropic.com/news/claude-s-model-spec>, 2023.
- Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024.
- Harrison Chase. LangChain. <https://github.com/langchain-ai/langchain>, 2022.
- Carson Denison, Monte MacDiarmid, Fazl Barez, David Hershfield, Shauna Kravec, Shannon Telleen-Lawton, Jennifer Siao, Akosua Busia, Evan Hubinger, Nicholas Schiefer, Jared Kaplan, Sam Bowman, Ethan Perez, and Stuart Ritchie. Sycophancy to subterfuge: Investigating reward tampering in language models. *arXiv preprint arXiv:2406.10162*, 2023.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- LangChain AI. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. 36, 2023.
- João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://github.com/joaomdmoura/crewAI>, 2024.

- Yohei Nakajima. BabyAGI. <https://github.com/yoheinakajima/babyagi>, 2023.
- Toran Bruce Richards et al. AutoGPT: An autonomous GPT-4 experiment. <https://github.com/Significant-Gravitas/AutoGPT>, 2023.
- Sanjit A. Seshia, Dorsa Sadigh, and Shankar Sastry. Formal specification for deep neural networks. In *Automated Technology for Verification and Analysis*, pages 20–34. Springer, 2018.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. 36, 2023.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. 36, 2023.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. 2023.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. 36, 2023.